

Replication: Design and Evaluation of Optimal Free Trials

Yoganarasimhan, Barzegary, and Pani (Management Science, 2023)

Weiheng Zhang

September 05, 2026

Contents

1	The paper in one page	2
1.1	What the replication package contains	2
2	A population calibrated to the paper	3
2.1	The simulator	3
2.2	Targets and achieved moments	4
2.3	The paper’s descriptive tables	4
3	Average effects and mechanism	5
3.1	Average treatment effects	5
3.2	Randomization check	6
3.3	Heterogeneity by region, skill, and job	6
3.4	Mechanism: usage and dormancy	7
4	Policy design	8
4.1	The authors’ lasso policy	8
4.2	Alternative outcome models	9
4.3	Treatment-effect models	10
4.4	What each method assigns	10
5	Policy evaluation	12
5.1	The IPS reward and the package’s off-policy value	12
5.2	Table 8: the value of each policy	13
5.3	Is the lasso policy better than 7-for-all?	13
6	Long-run outcomes	14
6.1	Average effects on subscription length and revenue	14
6.2	The subscription policy evaluated on long-run outcomes	14
6.3	Policies optimized on long-run outcomes	15
7	The shipped data, once more	16
8	Package tools beyond the paper	17
8.1	Validating a CATE model before building a policy on it	17
8.2	A depth-two policy tree on three arms	18
9	Lessons	19
10	References	19

1 The paper in one page

The question is how long a free trial should be, and whether the answer should differ across users. A SaaS firm that sold a 30-day trial ran a field experiment on 337,724 new users, randomizing them to 7-, 14-, or 30-day trials (15 / 15 / 70 percent), and followed subscriptions, subscription length, and revenue for two years. Six categorical variables describe each user before the trial: region, operating system, sign-up channel, job, skill, and business segment.

The average answer is surprising: the shortest trial wins. Seven days raises subscriptions by about 5 percent over 30 days (Table 5 of the paper), and also retention and revenue (Tables 9 and 10). The authors spend Sections 4 and 6 on the mechanism, because the theory literature predicts the opposite: longer trials increase usage, but they increase dormancy at the end of the trial even more, and dormant users do not subscribe (Figures 2 and 3, Tables 6 and 7).

The personalized answer is built from an outcome model and evaluated with an unbiased reward. A lasso of the outcome on covariate dummies, arm dummies, and their interactions predicts each user's outcome under each arm; the policy assigns the arm with the highest prediction (equation 3). Any policy is then priced on a held-out test set with the inverse-propensity-score (IPS) reward

$$\hat{R}_{\text{IPS}}(\pi) = \frac{1}{N} \sum_{i=1}^N \frac{\mathbb{1}\{W_i = \pi(X_i)\} Y_i}{\hat{e}_{\pi(X_i)}(W_i)},$$

which is unbiased because the experiment fixes the propensities. The lasso policy gains 6.8 percent over 30-for-all and beats the best uniform policy, 7-for-all (Table 8).

The paper's second message is a warning. Policies from linear regression and random forests look best on the training data and worst on the test data; CART and causal trees collapse to 7-for-all; causal forests find too little heterogeneity (Table 8, Figure 5). Predictive accuracy is not policy quality, and a uniform policy is a strong benchmark.

1.1 What the replication package contains

The public data carry no information: every column was replaced by independent random draws. The authors' `dataRandomizer.py` overwrites each column except the user id with uniform random integers. The check below, on the shipped `cleanDataset.csv`, shows a subscription rate of one half in every arm, equal arm shares, and no relationship between any two variables. The paper's numbers cannot be reproduced from these files, and neither can its descriptives.

```
shipped <- read.csv(file.path(data_path, "cleanDataset.csv"))
shipped_check <- tibble(
  quantity = c("Users", "Subscription rate", "Share in 7-day arm", "Share in 30-day arm",
    "R2 of subscription on arm and covariates", "Largest |correlation| among outcomes and arm"),
  paper = c("337,724", "0.148", "0.151", "0.698", "small but positive", "positive"),
  shipped = c(fmt0(nrow(shipped)), fmt(mean(shipped$subscribed)), fmt(mean(shipped$trial_length == 0)),
    fmt(mean(shipped$trial_length == 2)),
    fmt(summary(lm(subscribed ~ factor(trial_length) + factor(country) + factor(skill) + factor(job), data = shipped))$
      fmax(abs(cor(shipped[, c("subscribed", "total_revenue", "total_month", "trial_length")])[upper.tri(diag(4))])),
    )
```

What the package does give, completely, is the procedure. The Python and R scripts specify every table and figure, the IPS estimator, the argmax rule, the lasso specification with inverse-share weights and repeated cross-validation, and the causal-forest setup with the propensity fixed at the arm share. Two of the paper's arguments do not depend on the firm's data at all: the IPS reward is unbiased under randomization, and a policy from an overfitted outcome model looks best on training data and worst on test data. Those are the arguments this note reproduces.

Strategy. Section 2 builds a population that matches every moment the paper publishes (sample size, arm shares, covariate structure, subscription rates, the segment story of Section 6, the usage regressions of Table 6, and the long-run outcomes), with the true potential outcomes of every user known. The paper's procedure and our package are then run on identical inputs: the authors' estimators are matched numerically, each

Table 1: The shipped data after the authors' randomization: no moment of the paper survives.

Quantity	Paper	Shipped data
Users	337,724	337,724
Subscription rate	0.148	0.500
Share in 7-day arm	0.151	0.333
Share in 30-day arm	0.698	0.334
R2 of subscription on arm and covariates	small but positive	0.00010
Largest correlation among outcomes and arm	positive	0.0015

table is laid out as in the paper with the truth as an extra column, and the shipped data are run once more at the end to show what noise returns.

2 A population calibrated to the paper

2.1 The simulator

Each user has a subscription probability under each of the three arms, and everything else is generated around it. The baseline probability depends on region, operating system, channel, and job (and on a latent engagement that also drives usage). The arm effects follow the paper's Section 6: beginners and students respond best to 7 days, mixed-skill users and business professionals in the largest region to 14 days, experienced users to 30 days. Usage variables are drawn by arm and skill so that Table 6's regressions reappear; subscription length and revenue follow the paper's decomposition (equations 5 and 6): months conditional on subscribing, times a monthly price that is lower for students.

```
simulate_free_trial <- function(n, seed, par = list()) {
  set.seed(seed)
  p <- modifyList(list(
    base = -2.0,
    d7 = c(beginner = 0.10, mixed = -0.05, experienced = -0.12, unknown = 0.05, intermediate = 0.02),
    d14 = c(beginner = 0.00, mixed = 0.15, experienced = -0.08, unknown = 0.00, intermediate = 0.03),
    business14 = 0.13, student7 = 0.17, lat_sub = 0.35, lat_use = 0.25
  ), par)
  arm <- sample(arms, n, TRUE, c(0.1511, 0.1511, 0.6978))
  region <- sample(1:6, n, TRUE, c(55.02, 13.66, 9.12, 8.83, 8.0, 5.37))
  os <- sample(1:8, n, TRUE, c(28.97, 21.4, 14.04, 13.98, 9, 6, 4, 2.61))
  channel <- sample(1:42, n, TRUE, c(81.56, 8.14, 3.47, 0.81, rep(6.02 / 38, 38)))
  job_lab <- c("student", "business", "hobbyist", "designer", "photographer", "educator", "developer",
    "manager", "marketer", "artist", "engineer", "writer", "other", "unknown")
  job <- sample(job_lab, n, TRUE, c(28.20, 21.90, 20.34, 8.46, 4, 3, 3, 2.5, 2, 1.5, 1.2, 1, 1, 1.9))
  skill <- sample(c("beginner", "mixed", "experienced", "unknown", "intermediate"), n, TRUE,
    c(69.05, 12.75, 10.77, 7.38, 0.05))
  segment <- sample(1:7, n, TRUE, c(35.41, 32.74, 18.40, 7.81, 3, 1.8, 0.84))
  latent <- rnorm(n)
  base_x <- p$base + 0.15 * (region == 1) - 0.1 * (region == 4) + 0.08 * (os %in% 1:2) +
    0.25 * (job == "business") - 0.15 * (job == "student") + 0.1 * (job == "designer") - 0.1 * (channel == 2) +
    0.2 * (skill == "mixed") + 0.15 * (skill == "experienced") + p$lat_sub * latent
  delta7 <- p$d7[skill] + p$student7 * (job == "student" & skill == "experienced")
  delta14 <- p$d14[skill] + p$business14 * (job == "business" & region == 1)
  pr_mat <- cbind(plogis(base_x + delta7), plogis(base_x + delta14), plogis(base_x))
  colnames(pr_mat) <- paste0("pr_", arms)
  act_mean <- list(beginner = c(1.55, 1.95, 2.55), mixed = c(2.4, 3.9, 6.2), experienced = c(2.0, 2.9, 4.4),
    intermediate = c(1.9, 2.7, 3.9), unknown = c(1.7, 2.3, 3.2))
  am <- t(sapply(skill, function(s) act_mean[[s]])) * exp(p$lat_use * latent)
  u_use <- runif(n); u_pois <- runif(n); e_log <- rnorm(n, 0, 2.3); u_beta <- rbeta(n, 2.2, 1.05)
  usage <- lapply(seq_along(arms), function(k) {
    len <- arms[k]
    use <- as.integer(u_use < 0.828 + c(0, 0.004, 0.009)[k] + 0.03 * latent)
    active <- pmin(use * (1 + qpois(u_pois, pmax(am[, k] - 1, 0.05))), len)
    logu <- ifelse(use == 1, pmax(0, 4.35 + 0.24 * active + e_log), 0)
  })
}
```

```

dorm <- ifelse(use == 1, round(len * u_beta * (1 - 0.15 * pmin(pmax(latent, -2), 2))), len)
list(use = use, active = active, logu = logu, dorm = pmin(pmax(dorm, 0), len))
})
k <- match(arm, arms)
pick <- function(field) { M <- sapply(usage, `[`, field); M[cbind(seq_len(n), k)] }
sub_mat <- (runif(n) < pr_mat) * 1L
subscribed <- sub_mat[cbind(seq_len(n), k)]
months <- ifelse(subscribed == 1, pmin(round(rgamma(n, shape = 3.6, scale = 4.2) + 0.3 * (arm == 7)), 108), 0)
price <- exp(rnorm(n, log(33), 0.45)) * ifelse(job == "student", 0.65, 1) * ifelse(segment == 1, 1.15, 1)
revenue <- ifelse(subscribed == 1, round(months * price, 2), 0)
d <- data.frame(user_id = seq_len(n), arm, region, os, channel, job, skill, segment,
               use_software = pick("use"), active_days = pick("active"), log_usage = pick("logu"),
               dormancy = pick("dorm"), subscribed, months, revenue)
d <- cbind(d, pr_mat)
d$best_arm <- arms[max.col(pr_mat)]
d
}
pop <- simulate_free_trial(n_users, sim_seed)
pop$training <- as.integer(runif(n_users) < 0.7)
train <- pop[pop$training == 1, ]
test <- pop[pop$training == 0, ]

```

2.2 Targets and achieved moments

Table 2 lists every published moment the simulator was tuned to and what the simulated population delivers. The point of the table is honesty about what “calibrated” means: the arm shares, covariate shares, and usage regressions are matched by construction; the subscription rates, effects, segment shares, and long-run outcomes are matched by the choice of a dozen parameters and are close but not exact.

Table 2: Published moments used to calibrate the simulated population, and the values the simulation delivers. The best-arm shares are the shares of users whose true subscription probability is highest under each arm; the paper reports the shares its lasso policy assigns.

Moment	Paper	Simulated
Users	337,724	337,724
Share 7 / 14 / 30 days	15.1 / 15.1 / 69.8	15.1 / 15.0 / 69.9
Subscription rate	0.148	0.146
7-day minus 30-day subscription rate	0.006 to 0.008	0.0070
14-day minus 30-day subscription rate	0.002 to 0.005	0.0040
Share of users whose best arm is 7 / 14 / 30	68.9 / 23.2 / 7.9 (lasso policy)	70.2 / 23.3 / 6.5
Months subscribed, subscribers (mean, sd)	16.02, 8.43	15.22, 8.02
Revenue, subscribers (mean, sd)	568, 552	538, 421
Revenue, all users	79.1	78.8
Active days: intercept, 14-day, 30-day	1.75, 0.63, 1.71	1.50, 0.52, 1.34
Dormancy: intercept, 14-day, 30-day	4.63, 5.14, 16.43	5.05, 5.05, 16.57

2.3 The paper’s descriptive tables

Tables 1 and 2 of the paper establish the design before any analysis: who got which arm and how the covariates are distributed. Table 1 is read down the columns: shares of observations and of subscriptions by arm track each other, which is the first sign of small effects. Table 2 reports only the number of subcategories and the top four shares, because forty-two channels do not fit on a page.

Table 3 of the paper reports the outcomes for all users and for subscribers separately, because the long-run outcomes are zero for the 85 percent who never subscribe. The subscriber rows are the ones that carry the retention and revenue information.

Table 3: Summary statistics of treatment assignment and subscription rates in the simulated population (the layout of the paper’s Table 1).

	7-day trial	14-day trial	30-day trial	Total
Number of observations (N)	51,035	50,726	235,963	337,724
Percent of total observations	15.11	15.02	69.87	100
Number of subscriptions	7,743	7,590	34,103	49,436
Percent of total subscriptions	15.66	15.35	68.98	100
Subscription rate within group (in %)	15.17	14.96	14.45	14.64

Table 4: Summary statistics for the pretreatment categorical variables (the layout of the paper’s Table 2): number of subcategories and shares of the four largest.

Variable	Number of subcategories	Share of top subcategories			
		First	Second	Third	Fourth
Geographic region	6	54.98%	13.64%	9.13%	8.84%
Operating system	8	28.91%	21.42%	14.07%	14.01%
Signup channel	42	81.46%	8.18%	3.44%	0.83%
Job	14	28.23%	21.83%	20.46%	8.41%
Skill	5	68.99%	12.81%	10.76%	7.40%
Business segment	7	35.46%	32.66%	18.45%	7.82%

Table 5: Summary statistics of subscription, subscription length, and revenue outcomes (the layout of the paper’s Table 3).

Variable	Mean	Standard deviation	Minimum	25%	50%	75%	Maximum	Number of observations
Subscription	0.15	0.35	0.00	0.00	0.00	0.00	1.00	337,724
Subscription length (all)	2.23	6.19	0.00	0.00	0.00	0.00	74.00	337,724
Subscription length (subscribers)	15.22	8.02	0.00	9.00	14.00	20.00	74.00	49,436
Revenue (all)	78.80	249.39	0.00	0.00	0.00	0.00	8,129.76	337,724
Revenue (subscribers)	538.32	421.33	0.00	255.40	424.27	691.56	8,129.76	49,436

3 Average effects and mechanism

3.1 Average treatment effects

Table 5 of the paper is a difference in means with a t-statistic and a percentage gain over the 30-day baseline, reported for training and test data separately. In a randomized experiment nothing more is needed (their footnote 5: regression-adjusted estimates are indistinguishable). The split into training and test data appears already here because every later policy comparison is made on the test data, and the reader must know the test-set baseline.

```
ate_panel <- function(d, var, digits = 4, mult = 1) {
  ctrl <- d[[var]][d$arm == 30] * mult
  bind_rows(lapply(arms, function(a) {
    y <- d[[var]][d$arm == a] * mult
    tt <- if (a == 30) NULL else t.test(y, ctrl)
    tibble(Treatment = paste0(a, " days"), rate = mean(y),
           diff = if (a == 30) NA else mean(y) - mean(ctrl),
           t = if (a == 30) NA else unname(tt$statistic),
           gain = if (a == 30) NA else 100 * (mean(y) - mean(ctrl)) / mean(ctrl))
  })))
}
t5 <- bind_rows(Training = ate_panel(train, "subscribed"), Test = ate_panel(test, "subscribed"), .id = "Data")
```

The doubly robust score with the known propensity gives the same answer with a covariate

Table 6: Average effect of the 7- and 14-day treatments on subscription compared with the control condition of the 30-day free trial (the layout of the paper’s Table 5).

Data	Treatment	Subscription rate	Subscription rate difference	t statistics	Percentage gain over baseline
Training	7 days	0.1523	0.0078	3.74	5.41
	14 days	0.1494	0.0048	2.32	3.34
	30 days	0.1445	—	—	—
Test	7 days	0.1503	0.0057	1.81	3.97
	14 days	0.1503	0.0057	1.80	3.97
	30 days	0.1445	—	—	—

adjustment. The experiment fixes $e_k = P(W = k)$, so `dr_scores()` with the arm shares as propensities returns a score matrix whose column means are the expected outcomes under each arm; their differences are the ATEs of Table 6 with a regression adjustment for the covariates, which changes the estimates by less than a standard error, as the paper’s footnote 5 reports for its regression-adjusted version.

```
design_e <- function(d) matrix(rep(prop.table(table(factor(d$arm, levels = arms))), each = nrow(d)), nrow(d),
                             dimnames = list(NULL, arms))
add_dummies <- function(d) {
  mm <- model.matrix(~ factor(region) + factor(os) + factor(channel) + factor(job) + factor(skill) + factor(segment), d)[, -1]
  colnames(mm) <- paste0("z", seq_len(ncol(mm)))
  cbind(d, mm)
}
train_d <- add_dummies(train)
test_d <- add_dummies(test)
zcols <- grep("^z", names(train_d), value = TRUE)
sc_train <- dr_scores(train_d, "subscribed", "arm", zcols, arms = arms, p_hat = design_e(train_d),
                     learner_mu = lrn("regr.lm"), folds = 2, seed = 1)
sc_test <- dr_scores(test_d, "subscribed", "arm", zcols, arms = arms, p_hat = design_e(test_d),
                     learner_mu = lrn("regr.lm"), folds = 2, seed = 2)
sc_test$ate$contrasts
#> contrast estimate std.error
#> 1 14 - 7 -0.0002982107 0.004101974
#> 2 30 - 7 -0.0058705904 0.003186359
```

3.2 Randomization check

The paper regresses the arm on all covariate dummies and tests them jointly, rather than showing a table of means, because six categorical variables with up to 42 levels do not fit in a balance table. The F test below does the same on the simulated population for the 7-day and the 14-day indicators; the arms were assigned at random, so the covariates predict neither at the 5 percent level.

```
pop_d <- add_dummies(pop)
sapply(c(`7-day arm` = 7, `14-day arm` = 14), function(a) {
  f <- summary(lm(I(pop_d$arm == a) ~ as.matrix(pop_d[, zcols])))$fstatistic
  c(F = unname(f[1]), p = unname(pf(f[1], f[2], f[3], lower.tail = FALSE)))
})
#> 7-day arm 14-day arm
#> F 1.208608 1.0499828
#> p 0.103998 0.3605614
```

3.3 Heterogeneity by region, skill, and job

Figure 4 of the paper shows subscription rates by arm within categories to motivate personalization before any model is fitted. Bars by arm inside each category, with stars where the 7- or 14-day rate differs from the 30-day rate, let the reader see that beginners respond to 7 days and mixed-skill users to 14 days. The reader also sees the category shares under the bars, because a segment that responds differently only matters if it is large.

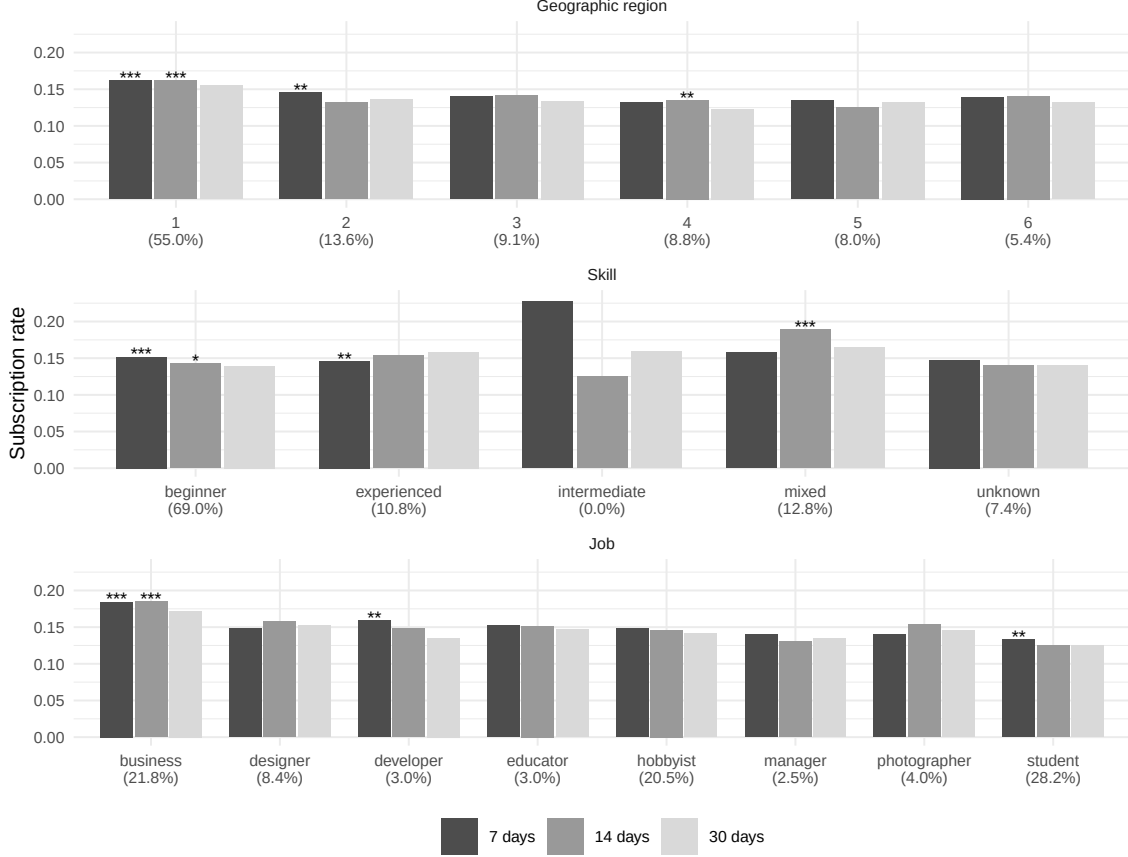


Figure 1: Heterogeneity in users' response to the three trial lengths within region, skill, and job (the layout of the paper's Figure 4). Bars are subscription rates by arm; stars mark 7- or 14-day rates that differ from the 30-day rate in the same category (* $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$); the share of users in each category is printed under its label.

3.4 Mechanism: usage and dormancy

Table 6 of the paper regresses each usage variable on the arm to show that longer trials raise usage, but sublinearly, and raise dormancy sharply. Because the arm is randomized these regressions are causal, so the table is the causal half of Figure 3; the second half, from usage to subscription, is only an association, which the paper marks by presenting Table 7 as suggestive (its footnote 8).

Table 7: Regression of usage features on trial length (the layout of the paper's Table 6). Standard errors in parentheses.

Outcome variable	Intercept	14-day trial	30-day trial	R2	N
Indicator for software use	0.831 (0.002)	0.002 (0.002)	0.006 (0.002)	0.000	337,724
Number of active days	1.499 (0.010)	0.525 (0.013)	1.343 (0.010)	0.055	337,724
Log usage during trial	3.994 (0.013)	0.129 (0.018)	0.342 (0.014)	0.002	337,724
Dormancy length	5.051 (0.028)	5.054 (0.040)	16.566 (0.031)	0.514	337,724

Figure 2 of the paper plots the subscription rate against activity for each arm, and the pattern it wants the reader to see is that at the same activity level shorter trials convert better. The left panel uses the number of active days; the right panel uses the last active day, so that dormancy, which is the trial length minus the last active day, is visible as distance from the right end of each curve.

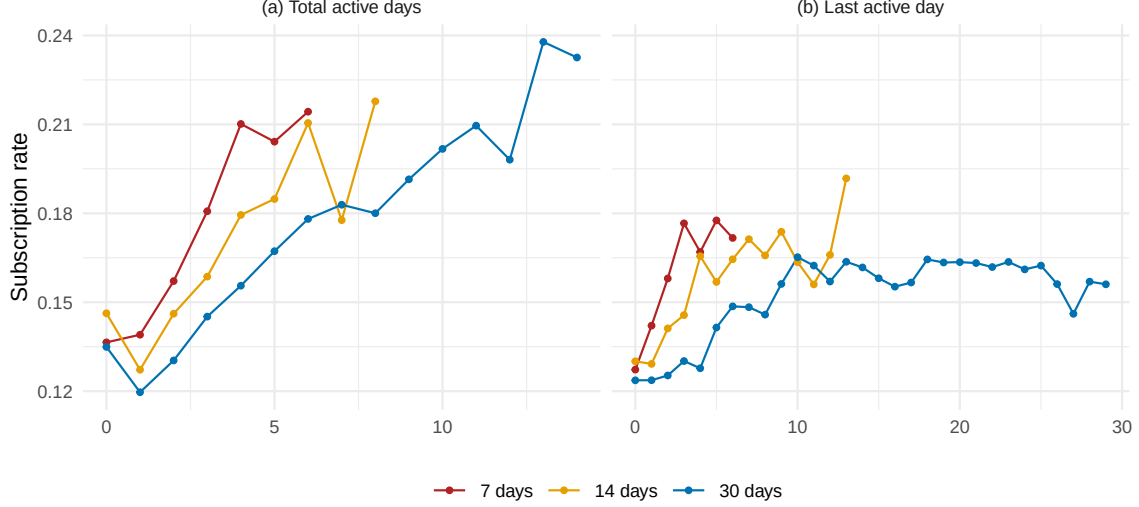


Figure 2: Relationship between usage and subscription rate for the three trial lengths (the layout of the paper’s Figure 2). Left: by the number of active days. Right: by the last active day (trial length minus dormancy).

Table 7 of the paper is a logit of subscription on the arm, the usage variables, and the covariates, and its purpose is to show that dormancy has a negative coefficient once usage is held fixed. With usage held fixed the arm coefficients turn positive for the longer trials: the longer trial is not harmful in itself, its dormancy is.

Table 8: Logit of subscription on trial length and usage features, with region, skill, job, and operating system as controls (the layout of the paper’s Table 7; the covariate rows are omitted as in the paper).

Variable	Coefficient	Standard error	z	p
14-day trial	0.0173	0.0182	0.95	0.343
30-day trial	0.0807	0.0205	3.93	0.000
Number of active days	0.0687	0.0030	23.25	0.000
Log usage during trial	0.0008	0.0023	0.34	0.730
Dormancy length	-0.0145	0.0009	-16.83	0.000
Indicator for software use	-0.2407	0.0199	-12.10	0.000

4 Policy design

4.1 The authors’ lasso policy

The paper’s policy is an outcome model, not a treatment-effect model: predict the subscription probability under each arm and assign the best arm (equation 3). The model is a lasso of the outcome on arm dummies, covariate dummies, and all arm-by-covariate interactions, so that each arm can have its own coefficient on each category. Two details matter. Observations are weighted by the inverse of their arm share, because the 30-day arm has five times more users and would otherwise dominate the interaction coefficients of the other two arms. The penalty is chosen by five-fold cross-validation and the whole procedure is repeated (50 times in the paper), because the selected penalty varies across folds; the paper reports the run with the median test-set value.

```
to_factors <- function(d) {
  for (v in covs) d[[v]] <- factor(d[[v]], levels = sort(unique(pop[[v]])))
  d$arm <- factor(d$arm, levels = arms)
```



```

d
}
lasso_formula <- ~ arm * (region + os + channel + job + skill + segment) - 1
lasso_X <- function(d) Matrix::sparse.model.matrix(lasso_formula, to_factors(d))
arm_share <- function(d) prop.table(table(factor(d$arm, levels = arms)))
train$e <- as.numeric(arm_share(train)[as.character(train$arm)])
test$e <- as.numeric(arm_share(test)[as.character(test$arm)])
predict_arms <- function(pred_fun, d) {
  out <- sapply(arms, function(a) { d$arm <- a; pred_fun(d) })
  colnames(out) <- arms
  out
}
argmax_policy <- function(pred) arms[max.col(pred, ties.method = "first")]
fit_lasso_policy <- function(train, test, y = "subscribed", reps = 1, seed = 1) {
  Xtr <- lasso_X(train)
  lapply(seq_len(reps), function(r) {
    set.seed(seed + r)
    cv <- cv.glmnet(Xtr, train[[y]], weights = 1 / train$e, alpha = 1, nfolds = 5, type.measure = "mse")
    fit <- glmnet(Xtr, train[[y]], weights = 1 / train$e, alpha = 1, lambda = cv$lambda.min)
    pf <- function(d) as.numeric(predict(fit, lasso_X(d)))
    list(lambda = cv$lambda.min, n_nonzero = sum(coef(fit) != 0),
         pred_train = predict_arms(pf, train), pred_test = predict_arms(pf, test))
  })
}

lasso_runs <- fit_lasso_policy(train, test, reps = params$n_lasso_reps, seed = 100)

```

Repeated cross-validation selects a slightly different penalty each time, and the paper reports the median run to avoid picking a lucky one. The runs below differ in the penalty and in the number of nonzero coefficients; the test-set value of each run’s policy is computed in Section 5 and the median run is carried forward.

Table 9: Repeated cross-validated lasso fits of the authors’ specification.

Run	Selected penalty	Nonzero coefficients	Share assigned 7 days (test)
1	0.000855	77	0.652
2	0.000855	77	0.652
3	0.001030	69	0.652
4	0.000855	77	0.652
5	0.001030	69	0.652

4.2 Alternative outcome models

The paper fits the same outcome model with four other learners, to show that the choice of learner is a choice of policy. Linear regression is the lasso without the penalty. CART is a single regression tree pruned by cross-validation. The random forest and XGBoost use the authors’ tuned settings (1,000 trees with a minimum leaf of 70; boosted trees of depth 6 with the logistic objective). All four get the arm as a feature together with the covariate dummies and predict each user’s outcome under each arm.

```

feat <- c("arm", zcols)
mlr3_policy <- function(learner, train_d, test_d) {
  d <- train_d[, c("subscribed", feat)]
  task <- as_task_regr(d, target = "subscribed")
  learner$train(task)
  pf <- function(dd) learner$predict_newdata(dd[, feat])$response
  list(pred_train = predict_arms(pf, train_d), pred_test = predict_arms(pf, test_d))
}
set.seed(1)
fit_reg <- {
  m <- lm(update(lasso_formula, subscribed ~ .), data = to_factors(train), weights = 1 / e)
  pf <- function(d) as.numeric(predict(m, newdata = to_factors(d)))
  list(pred_train = predict_arms(pf, train), pred_test = predict_arms(pf, test))
}

```

```

}
fit_cart <- {
  d <- to_factors(train)[, c("subscribed", "arm", covs)]
  tr <- rpart::rpart(subscribed ~ ., data = d, method = "anova", control = rpart::rpart.control(xval = 5, cp = 0, minbucket = 10))
  tr <- rpart::prune(tr, cp = tr$cptable[which.min(tr$cptable[, "xerror"]), "CP"])
  pf <- function(dd) as.numeric(predict(tr, newdata = to_factors(dd)[, c("arm", covs)]))
  list(pred_train = predict_arms(pf, train), pred_test = predict_arms(pf, test), tree = tr)
}
fit_rf <- mlr3_policy(lrn("regr.ranger", num.trees = n_trees, min.node.size = 70, num.threads = n_threads, seed = 1), train_d, test_d)
#> Growing trees.. Progress: 25%. Estimated remaining time: 1 minute, 31 seconds.
#> Growing trees.. Progress: 53%. Estimated remaining time: 55 seconds.
#> Growing trees.. Progress: 78%. Estimated remaining time: 25 seconds.
fit_xgb <- mlr3_policy(lrn("regr.xgboost", objective = "reg:logistic", eta = 0.4995, alpha = 0.7245, subsample = 0.8,
  colsample_bytree = 0.9, max_depth = 6, nrounds = 200, nthread = n_threads), train_d, test_d)

```

4.3 Treatment-effect models

The paper also derives policies from pairwise treatment-effect estimates, because the marketing literature increasingly uses causal trees and forests for targeting. For each pair of arms the training users of those two arms form a binary experiment with a known propensity (the ratio of the two shares), a causal forest estimates the pairwise effect, and the policy chooses the arm that wins its comparisons. The package builds the pairwise objects from the three-arm scores with `contrast_scores()`; the DR-learner and a pruned regression tree on the doubly robust score (the closest single-tree analogue of the paper's causal tree available without the archived `causalTree` package) are fitted on the same objects.

```

pairs <- list(c(7, 30), c(14, 30))
pair_scores <- lapply(pairs, function(p) contrast_scores(sc_train, treat = p[1], control = p[2]))
pair_test <- lapply(pairs, function(p) contrast_scores(sc_test, treat = p[1], control = p[2]))
rf_final <- lrn("regr.ranger", num.trees = n_trees, min.node.size = 200, num.threads = n_threads, seed = 1)
dr_models <- lapply(pair_scores, function(s) cate_learner(scores = s, x_het = zcols, method = "dr", learner_final = rf_final))
#> Growing trees.. Progress: 46%. Estimated remaining time: 35 seconds.
#> Growing trees.. Progress: 94%. Estimated remaining time: 3 seconds.
#> Growing trees.. Progress: 47%. Estimated remaining time: 35 seconds.
#> Growing trees.. Progress: 95%. Estimated remaining time: 3 seconds.
cf_models <- lapply(seq_along(pairs), function(j) {
  s <- pair_scores[[j]]
  grf::causal_forest(as.matrix(s$data[, zcols]), s$y, s$d, W.hat = s$nuisance$p, Y.hat = s$nuisance$l,
    num.trees = n_trees, honesty = TRUE, num.threads = n_threads, seed = 1)
})
ct_models <- lapply(pair_scores, function(s) {
  d <- to_factors(s$data[, covs]); d$score <- s$score
  tr <- rpart::rpart(score ~ ., data = d, method = "anova", control = rpart::rpart.control(xval = 5, cp = 0, minbucket = 500))
  tr <- rpart::prune(tr, cp = tr$cptable[which.min(tr$cptable[, "xerror"]), "CP"])
})
tau_pairs <- function(models, d, pred) cbind(`7` = pred(models[[1]], d), `14` = pred(models[[2]], d), `30` = 0)
pred_dr <- function(m, d) as.numeric(predict(m, d))
pred_cf <- function(m, d) as.numeric(predict(m, as.matrix(d[, zcols]))$predictions)
pred_ct <- function(m, d) as.numeric(predict(m, newdata = to_factors(d[, covs]))
fit_dr <- list(pred_train = tau_pairs(dr_models, train_d, pred_dr), pred_test = tau_pairs(dr_models, test_d, pred_dr))
fit_cf <- list(pred_train = tau_pairs(cf_models, train_d, pred_cf), pred_test = tau_pairs(cf_models, test_d, pred_cf))
fit_ct <- list(pred_train = tau_pairs(ct_models, train, pred_ct), pred_test = tau_pairs(ct_models, test, pred_ct))

```

4.4 What each method assigns

Table A8 of the paper reports the share of users each policy sends to each arm, and it is the fastest way to see that the methods disagree about who should get what. CART and the single tree on the score prescribe 7 days for everyone (the paper's footnote to Table 8), while the forests spread users across the three arms the most; the lasso sends about two thirds of users to 7 days and a quarter to 14 days, close to the paper's 68.9 and 23.2 percent.

```

lasso_pick <- 1L # replaced by the median run in the next section
fits <- list(reg = fit_reg, cart = fit_cart, r_forest = fit_rf, xgboost = fit_xgb, dr_learner = fit_dr, c_forest = fit_cf, c_tree = fit_ct)
policies_from <- function(fits, lasso_run, which = "pred_test", n) {
  c(list(`7` = rep(7, n), `14` = rep(14, n), `30` = rep(30, n), lasso = argmax_policy(lasso_run[[which]])),

```

```
lapply(fits, function(f) argmax_policy(f[[which]])))
}
```

Table 10: Share of users (in percent) assigned to each trial length by each policy (the layout of the paper’s Table A8). The lasso row uses the first cross-validation run here; the median run is selected in the next section.

Policy	Training set			Test set		
	7 days	14 days	30 days	7 days	14 days	30 days
π_{lasso}	65.07	25.56	9.36	65.17	25.57	9.26
π_{reg}	54.81	33.15	12.04	54.63	33.20	12.16
π_{cart}	100.00	0.00	0.00	100.00	0.00	0.00
π_{r_forest}	45.94	27.65	26.41	45.82	27.72	26.46
$\pi_{xgboost}$	49.91	27.59	22.50	49.63	27.63	22.74
$\pi_{dr_learner}$	41.61	36.14	22.25	41.83	36.05	22.12
π_{c_forest}	44.54	35.97	19.49	44.53	36.01	19.46
π_{c_tree}	100.00	0.00	0.00	100.00	0.00	0.00

Figure 5 of the paper plots the distribution of the estimated 7-versus-30 effect across test users for every method, because the amount of heterogeneity a method finds predicts how well its policy does. The paper’s reading is that CART, the causal tree, and the causal forest show almost no heterogeneity, so their policies cannot personalize, while the regression and random forest show the most and overfit. Here the true effect is known and drawn as the reference curve, so each method can be read as too flat or too spread relative to the truth; the table after the figure gives the spread of each curve and its correlation with the truth. Every learner except the lasso spreads its estimates two to five times more than the truth, the two trees show none, and the correlation with the truth ranks the methods broadly as the test-set values in Table 12 do.

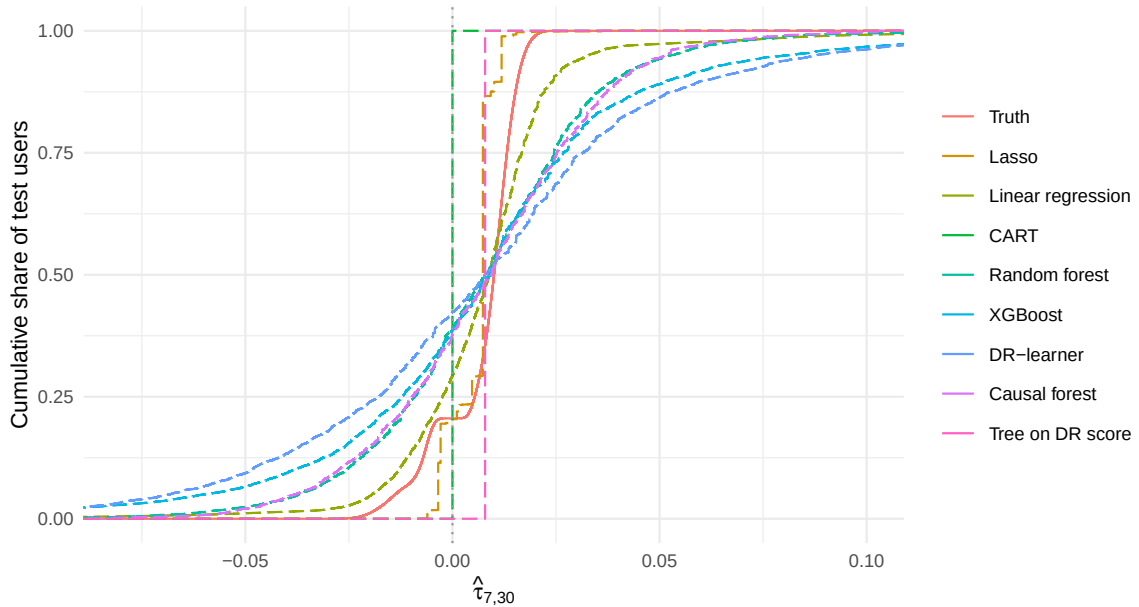


Figure 3: Cumulative distribution of the estimated effect of 7 versus 30 days across test users for each method (the layout of the paper’s Figure 5), with the true effect for reference.

Table 11: Spread of the estimated 7-versus-30 effect across test users by method, and its correlation with the true effect.

Method	Standard deviation	Share positive	Correlation with the truth
Truth	0.010	0.794	1.000
Lasso	0.005	0.796	0.771
Linear regression	0.022	0.708	0.375
CART	0.000	0.000	NA
Random forest	0.029	0.610	0.231
XGBoost	0.048	0.614	0.183
DR-learner	0.051	0.576	0.170
Causal forest	0.028	0.622	0.268
Tree on DR score	0.000	1.000	NA

5 Policy evaluation

5.1 The IPS reward and the package’s off-policy value

Every policy is priced with the same estimator on the same test set, because the paper’s argument is that estimators must be compared on the value of the policies they produce, not on their fit. The authors’ function (their `CalculatePolicyProfit`) keeps the users whose assigned arm equals the arm the policy would give them, and weights each by the inverse of the share of that arm among the users the policy sends there. This is the IPS reward with the propensity estimated within each policy group (their footnote 10). The package computes the same quantity when the multi-arm score object is built with `type = "ipw"` and those propensities; with the design shares instead, the estimates differ in the fourth decimal.

```
policy_profit <- function(d, policy, y = "subscribed") {
  keep <- d$arm == policy
  es <- table(factor(d$arm[keep], levels = arms)) / table(factor(policy, levels = arms))
  sum(d[[y]][keep] / as.numeric(es[as.character(d$arm[keep])])) / nrow(d)
}

ipw_scores <- function(d, y = "subscribed", policy = NULL) {
  p <- if (is.null(policy)) design_e(d) else {
    keep <- d$arm == policy
    es <- table(factor(d$arm[keep], levels = arms)) / table(factor(policy, levels = arms))
    matrix(rep(as.numeric(es), each = nrow(d)), nrow(d), dimnames = list(NULL, arms))
  }
  dr_scores(d, y, "arm", arms = arms, p_hat = p, type = "ipw", folds = 2)
}

truth_value <- function(d, policy) mean(as.matrix(d[, paste0("pr_", arms)])[cbind(seq_len(nrow(d)), match(policy, arms))])
pol_lasso_test <- argmax_policy(lasso_runs[[1]]$pred_test)
c(paper_function = policy_profit(test, pol_lasso_test),
  package_within_policy = policy_value(ipw_scores(test, policy = pol_lasso_test), pol_lasso_test)$estimate,
  package_design_shares = policy_value(ipw_scores(test), pol_lasso_test)$estimate,
  truth = truth_value(test, pol_lasso_test))
#>      paper_function package_within_policy package_design_shares
#>      0.1540870      0.1540870      0.1534727
#>      truth
#>      0.1564101
```

The median lasso run is selected on the test-set IPS value, as in the paper.

```
lasso_test_values <- sapply(lasso_runs, function(r) policy_profit(test, argmax_policy(r$pred_test)))
lasso_pick <- order(lasso_test_values)[ceiling(length(lasso_test_values) / 2)]
round(lasso_test_values, 5); lasso_pick
#> [1] 0.15409 0.15409 0.15425 0.15409 0.15425
#> [1] 4
```

5.2 Table 8: the value of each policy

```

pols_train <- policies_from(fits, lasso_runs[[lasso_pick]], "pred_train", nrow(train))
pols_test  <- policies_from(fits, lasso_runs[[lasso_pick]], "pred_test", nrow(test))
ips_train  <- sapply(pols_train, function(p) policy_profit(train, p))
ips_test   <- sapply(pols_test, function(p) policy_profit(test, p))
dr_val     <- policy_value(sc_test, pols_test, baseline = "30")
truth_val  <- sapply(pols_test, function(p) truth_value(test, p))
t8 <- tibble(policy = names(pols_test), ips_train = ips_train, ips_test = ips_test,
             gain_train = 100 * (ips_train / ips_train[["30"]] - 1), gain_test = 100 * (ips_test / ips_test[["30"]] - 1),
             dr_test = dr_val$estimate + dr_val$baseline_value, dr_gain = dr_val$gain_pct,
             true_test = truth_val, true_gain = 100 * (truth_val / truth_val[["30"]] - 1))

```

Table 8 of the paper has two blocks, training and test, and two numbers per block, the estimated subscription rate and its percentage gain over 30-for-all; the reader is meant to compare the two blocks row by row. The lasso policy beats every uniform policy on the test set, with a test gain of 6.62 percent against the paper’s 6.81. The forest-based policies (random forest, DR-learner, causal forest) and XGBoost show the largest training gains and the smallest test gains, which is the overfitting warning; the paper’s random forest shows the same reversal. CART and the tree on the score collapse to 7-for-all, so their rows repeat the 7-for-all row. The two column pairs added here, the doubly robust value and the true value, are available only in a simulation and confirm that the IPS test-set numbers rank the policies correctly: the lasso is first on all three, and the largest gap between a test-set IPS estimate and the truth is 0.23 percentage points.

Table 12: Gains in subscription from implementing different counterfactual free-trial policies (the layout of the paper’s Table 8). The first four numeric columns are the IPS estimates as in the paper; the doubly robust columns use the regression-adjusted scores; the true columns average each user’s true subscription probability under the arm the policy assigns. Subscription rates and increases are in percent; increases are relative to the 30-day-for-all policy.

Policy category	Policy	Estimated subscription (%)		Increase in subscription (%)		Doubly robust, test set		True value, test set	
		Training set	Test set	Training set	Test set	Estimate	Increase	Estimate	Increase
Personalized based on lasso	π_{lasso}	15.96	15.41	10.46	6.62	15.40	6.57	15.64	8.16
Uniform	π_7	15.23	15.03	5.41	3.97	15.03	4.06	15.16	4.82
	π_{14}	14.94	15.03	3.34	3.97	15.00	3.86	14.87	2.80
	π_{30}	14.45	14.45	—	—	14.45	—	14.46	—
Alternative personalized policies	π_{reg}	16.23	15.28	12.26	5.71	15.25	5.57	15.40	6.51
	π_{cart}	15.23	15.03	5.41	3.97	15.03	4.06	15.16	4.82
	π_{r_forest}	23.25	14.89	60.87	3.05	14.91	3.22	15.09	4.32
	$\pi_{xgboost}$	18.13	15.14	25.43	4.79	15.14	4.82	15.20	5.14
	$\pi_{dr_learner}$	24.35	14.94	68.46	3.39	14.96	3.56	15.06	4.14
	π_{c_forest}	20.32	15.00	40.59	3.77	14.98	3.70	15.14	4.68
	π_{c_tree}	15.23	15.03	5.41	3.97	15.03	4.06	15.16	4.82

The training-versus-test reversal is the paper’s central caution and it reappears here, in a stronger form. The forests and XGBoost predict their own training rows in sample, so the argmax rule follows the noise in those rows and the training IPS value rewards it, because the same rows carry the realized outcomes. The training gains of these methods are far above anything a policy can deliver (the truth column caps the gain near 8 percent), and their test gains are the smallest. The paper’s random forest shows the same pattern at a smaller scale (18.67 percent in training, 1.32 in test). The lasso’s penalty removes most interaction coefficients, so its training and test gains are close, and the true value confirms its test gain.

5.3 Is the lasso policy better than 7-for-all?

The paper answers with a bootstrap over test users, because the lasso and the 7-for-all policies are evaluated on overlapping users and their difference needs a paired standard error. The same paired difference is available analytically: the multi-arm `policy_value()` reports each policy’s difference from the first policy in the list with the standard error of the per-user differences. The bootstrap reproduces the paper’s t statistic (3.12 in the paper). The analytic standard errors are larger, for two reasons: the

Table 13: Overfitting of the policy: the difference between the training and test IPS gains, against the true gain.

Policy	Training gain (%)	Test gain (%)	Training minus test	True gain (%)
$\pi_{dr_learner}$	68.46	3.39	65.07	4.14
π_{r_forest}	60.87	3.05	57.82	4.32
π_{c_forest}	40.59	3.77	36.83	4.68
$\pi_{xgboost}$	25.43	4.79	20.64	5.14
π_{reg}	12.26	5.71	6.55	6.51
π_{lasso}	10.46	6.62	3.84	8.16
π_{cart}	5.41	3.97	1.45	4.82
π_{c_tree}	5.41	3.97	1.45	4.82

bootstrap re-estimates the within-policy propensities in every draw, which turns the IPS reward into a ratio estimator with a smaller variance than the per-user differences with fixed propensities, and ten bootstrap rounds estimate a standard error imprecisely. The true difference, known in the simulation, is positive and close to both point estimates.

```
set.seed(7)
boot_diff <- replicate(params$n_boot_rounds, {
  i <- sample.int(nrow(test), replace = TRUE)
  policy_profit(test[i, ], pols_test$lasso[i]) - policy_profit(test[i, ], pols_test[["7"]][i])
})
boot_t <- t.test(boot_diff)
paired <- policy_value(ipw_scores(test), list(`7` = pols_test[["7"]], lasso = pols_test$lasso, `14` = pols_test[["14"]]))
tibble(method = c(paste0("Bootstrap over test users (", params$n_boot_rounds, " rounds)"), "Paired per-user difference, IPS scores"),
  difference = c(mean(boot_diff), paired$diff_vs_first[2], policy_value(sc_test, list(`7` = pols_test[["7"]], lasso = pols_test$lasso, `14` = pols_test[["14"]])),
  std_error = c(sd(boot_diff), paired$diff_se[2], policy_value(sc_test, list(`7` = pols_test[["7"]], lasso = pols_test$lasso, `14` = pols_test[["14"]])),
  mutate(t = difference / std_error) %>%
  bind_rows(tibble(method = "True difference", difference = truth_val[["lasso"]] - truth_val[["7"]]))
#> # A tibble: 4 x 4
#>   method                                     difference std_error      t
#>   <chr>                                     <dbl>      <dbl> <dbl>
#> 1 Bootstrap over test users (10 rounds)      0.00507    0.00171  2.96
#> 2 Paired per-user difference, IPS scores      0.00322    0.00263  1.22
#> 3 Paired per-user difference, doubly robust scores 0.00363    0.00241  1.51
#> 4 True difference                          0.00483    NA      NA
```

6 Long-run outcomes

6.1 Average effects on subscription length and revenue

Tables 9 and 10 of the paper repeat the Table 5 layout for subscription length and revenue, because a policy that raises subscriptions could still lower revenue if the marginal subscribers are short-lived or cheap. The paper decomposes each long-run outcome into the subscription decision and the outcome conditional on subscribing (equations 5 and 6), and finds the conditional part almost flat across arms; the gains in retention and revenue are therefore driven by conversions.

6.2 The subscription policy evaluated on long-run outcomes

Table 11 of the paper evaluates the same four policies on subscription length and revenue with the same IPS estimator, changing only the outcome, to show that the policy designed on subscriptions also raises the long-run outcomes. The estimator does not need to be refitted: the policy is fixed and the reward function accepts any outcome. In the simulation the lasso policy and the 7-day uniform policy are within noise of each other on the long-run outcomes, because months and revenue are generated almost independently of the arm once a user subscribes and their variance is far larger than the difference between the two policies.

Table 14: Average effect of trial length on subscription length in months (the layout of the paper’s Table 9).

Data	Treatment	Subscription length	Difference	t statistics	Percentage gain over baseline
Training	7 days	2.329	0.132	3.59	5.98
	14 days	2.268	0.071	1.94	3.22
	30 days	2.198	—	—	—
Test	7 days	2.363	0.178	3.13	8.17
	14 days	2.295	0.110	1.98	5.05
	30 days	2.185	—	—	—

Table 15: Average effect of trial length on revenue in dollars (the layout of the paper’s Table 10).

Data	Treatment	Revenue	Difference	t statistics	Percentage gain over baseline
Training	7 days	82.38	4.66	3.12	6.00
	14 days	80.93	3.21	2.18	4.13
	30 days	77.72	—	—	—
Test	7 days	85.63	8.93	3.86	11.65
	14 days	80.00	3.30	1.51	4.30
	30 days	76.70	—	—	—

Table 16: The conditional part of the decomposition in the paper’s equations 5 and 6: subscription length and revenue among subscribers, by arm.

Arm	Months, subscribers	Revenue, subscribers	Revenue per month
7	15.42	549.43	35.63
14	15.21	539.00	35.43
30	15.18	535.64	35.29

Table 17: IPS estimates of the average subscription length and revenue under three uniform and one personalized policy (the layout of the paper’s Table 11).

Category	Policy	Subscription length				Revenue			
		Estimate (months)		Increase (%)		Estimate (\$)		Increase (%)	
		Training	Test	Training	Test	Training	Test	Training	Test
Personalized	π_{lasso}	2.41	2.36	9.83	8.02	85.94	84.92	10.57	10.71
Uniform	π_7	2.33	2.36	5.98	8.17	82.38	85.63	6.00	11.65
	π_{14}	2.27	2.29	3.22	5.05	80.93	80.00	4.13	4.30
	π_{30}	2.20	2.18	—	—	77.72	76.70	—	—

6.3 Policies optimized on long-run outcomes

Table 12 of the paper trains the same lasso on revenue and on subscription length and evaluates all three policies on all three outcomes, to answer whether a firm should wait two years to design its policy. The paper’s answer is no: the subscription policy does at least as well on every outcome, because the conditional outcomes hardly differ across arms, the long-run outcomes have far higher variance, and a binary outcome is easier to learn. The rows are the training outcome and the columns the evaluation outcome, so the diagonal is where each policy should shine if long-run training paid off. Here the three policies are within a few tenths of a percent of each other on every outcome; the revenue-trained policy is the worst on subscriptions and revenue, which is the paper’s variance argument, and no policy gains from

being trained on the outcome it is evaluated on.

```
lasso_months <- fit_lasso_policy(train, test, y = "months", reps = 1, seed = 200)[[1]]
lasso_revenue <- fit_lasso_policy(train, test, y = "revenue", reps = 1, seed = 300)[[1]]
```

Table 18: Expected mean of the three outcomes under policies optimizing each outcome (the layout of the paper’s Table 12).

Data set	Policy optimized on	Subscription (%)	Total revenue (\$)	Subscription length (months)
Training data	Subscription	15.96	85.94	2.41
	Total revenue	15.85	85.82	2.40
	Subscription length	15.86	85.51	2.41
Test data	Subscription	15.41	84.92	2.36
	Total revenue	15.38	84.07	2.36
	Subscription length	15.38	85.25	2.37

7 The shipped data, once more

Running the paper’s own procedure on the shipped file shows what the randomized data return: a subscription rate of one half under every policy and gains within a fraction of a percent of zero. The lasso is fitted on the shipped training rows with the authors’ specification, and each policy is priced with the authors’ IPS function. Because every column is independent noise, all policies have the same true value; the penalized fit keeps no interaction and the lasso policy collapses to a uniform policy.

```
sh <- shipped %>% transmute(user_id, arm = arms[trial_length + 1], region = country, os, channel = signup_app, job, skill, segme
  subscribed, months = ifelse(is.na(total_month), 0, total_month), revenue = total_revenue, training)
sh_train <- sh[sh$training == 1, ]; sh_test <- sh[sh$training == 0, ]
sh_train$e <- as.numeric(arm_share(sh_train)[as.character(sh_train$arm)])
sh_test$e <- as.numeric(arm_share(sh_test)[as.character(sh_test$arm)])
sh_to_factors <- function(d) { for (v in covs) d[[v]] <- factor(d[[v]], levels = sort(unique(sh[[v]]))); d$arm <- factor(d$arm,
sh_X <- function(d) Matrix::sparse.model.matrix(lasso_formula, sh_to_factors(d))
set.seed(11)
sh_cv <- cv.glmnet(sh_X(sh_train), sh_train$subscribed, weights = 1 / sh_train$e, alpha = 1, nfolds = 5)
sh_fit <- glmnet(sh_X(sh_train), sh_train$subscribed, weights = 1 / sh_train$e, alpha = 1, lambda = sh_cv$lambda.min)
sh_pf <- function(d) as.numeric(predict(sh_fit, sh_X(d)))
sh_pols <- list(train = argmax_policy(predict_arms(sh_pf, sh_train)), test = argmax_policy(predict_arms(sh_pf, sh_test)))
shipped_tab <- bind_rows(lapply(c(Training = "train", Test = "test"), function(s) {
  d <- if (s == "train") sh_train else sh_test
  pols <- list(lasso = sh_pols[[s]], `7` = rep(7, nrow(d)), `14` = rep(14, nrow(d)), `30` = rep(30, nrow(d)))
  v <- sapply(pols, function(p) policy_profit(d, p))
  tibble(policy = names(v), value = 100 * v, gain = 100 * (v / v[["30"]] - 1))
}), .id = "data")
```

Table 19: The paper’s Table 8 procedure applied to the shipped, randomized data: subscription rates and gains in percent.

Policy	Estimated subscription (%)		Increase in subscription (%)	
	Training set	Test set	Training set	Test set
π_{lasso}	49.64	49.91	-0.39	-0.26
π_7	49.64	49.91	-0.39	-0.26
π_{14}	50.21	50.13	0.74	0.17
π_{30}	49.84	50.05	—	—

8 Package tools beyond the paper

8.1 Validating a CATE model before building a policy on it

The paper judges each effect estimator only through the value of its policy; the package can also test the estimator directly on the test-set scores. `cate_validate()` takes a model of the 7-versus-30 effect and the test-set pair object and reports the calibration slope, the effect by quartile of the prediction, and the rank-weighted average effect (AUTOC) with a bootstrap standard error. A slope near one with a positive AUTOC says the model ranks users correctly, which is what a targeting policy needs from it. The DR-learner fails this test: its slope is near zero, the quartile with the highest predictions has no larger effect than the others, and the AUTOC is zero within its standard error. This is the same verdict Table 12 reached through the policy value, obtained before any policy is built. The lasso's own implied effect, the difference between its two arm predictions, passes: the slope is not distinguishable from one, the group effects rise with the prediction (the predictions take few distinct values, so only three groups form), and the QINI area is positive at the 1 percent level; the AUTOC, which weights the top ranks most, is positive at the 10 percent level.

```
val <- cate_validate(pair_test[[1]], tau_hat = dr_models[[1]], n_groups = 4, n_boot = 199, seed = 1)
val
#> Validation of a CATE model on 85959 held-out observations
#>
#> Heterogeneity test (OLS of the score on the centered prediction):
#>      term estimate std.error p.value   conf.low conf.high
#>      ATE 0.005861  0.003186 0.06581 -0.0003831  0.0121
#> tau_hat (centered) 0.003861  0.067426 0.95434 -0.1282923  0.1360
#>
#> Calibration by prediction quantile groups:
#> group   n mean_tau   gate std.error   conf.low conf.high
#>    1 21515 -0.050011  0.0035051  0.006391 -0.0090203  0.01603
#>    2 21465 -0.004933  0.0084627  0.006373 -0.0040281  0.02095
#>    3 21489  0.019522  0.0122717  0.006429 -0.0003288  0.02487
#>    4 21490  0.068284 -0.0007906  0.006292 -0.0131228  0.01154
#> call = 0.0358, normalized = -6.952; equal-effects test p = 0.491
#>
#> Areas under the targeting curves (one-sided 95% lower bounds):
#> statistic estimate std.error   conf.low p.value
#>    AUTOC -0.0011592  0.003330 -0.006637  0.6361
#>    AUQC  -0.0003659  0.000918 -0.001876  0.6549
#> largest lower simultaneous band: TOC 0 at q = 1; QINI 0 at q = 1
rows_7_30 <- test$arm %in% c(7, 30)
tau_lasso <- lasso_runs[[lasso_pick]]$pred_test[rows_7_30, "7"] - lasso_runs[[lasso_pick]]$pred_test[rows_7_30, "30"]
val_lasso <- cate_validate(pair_test[[1]], tau_hat = tau_lasso, n_groups = 4, n_boot = 199, seed = 1)
val_lasso
#> Validation of a CATE model on 85959 held-out observations
#>
#> Heterogeneity test (OLS of the score on the centered prediction):
#>      term estimate std.error p.value   conf.low conf.high
#>      ATE 0.005861  0.003186 0.065801 -0.0003829  0.0121
#> tau_hat (centered) 1.681324  0.652141 0.009933  0.4031518  2.9595
#>
#> Calibration by prediction quantile groups:
#> group   n mean_tau   gate std.error   conf.low conf.high
#>    1 24771 -0.001329 -0.01127  0.005902 -0.022837  0.000299
#>    2 47362  0.007332  0.01251  0.004306  0.004068  0.020947
#>    3 13826  0.011185  0.01378  0.007926 -0.001753  0.029318
#> call = 0.0061, normalized = 0.379; equal-effects test p = 0.00273
#>
#> Areas under the targeting curves (one-sided 95% lower bounds):
#> statistic estimate std.error   conf.low p.value
#>    AUTOC 0.004355  0.0031312 -0.0007955  0.082142
#>    AUQC  0.002581  0.0008456  0.0011906  0.001134
#> largest lower simultaneous band: TOC 0.0028 at q = 0.65; QINI 0.002 at q = 0.7
```

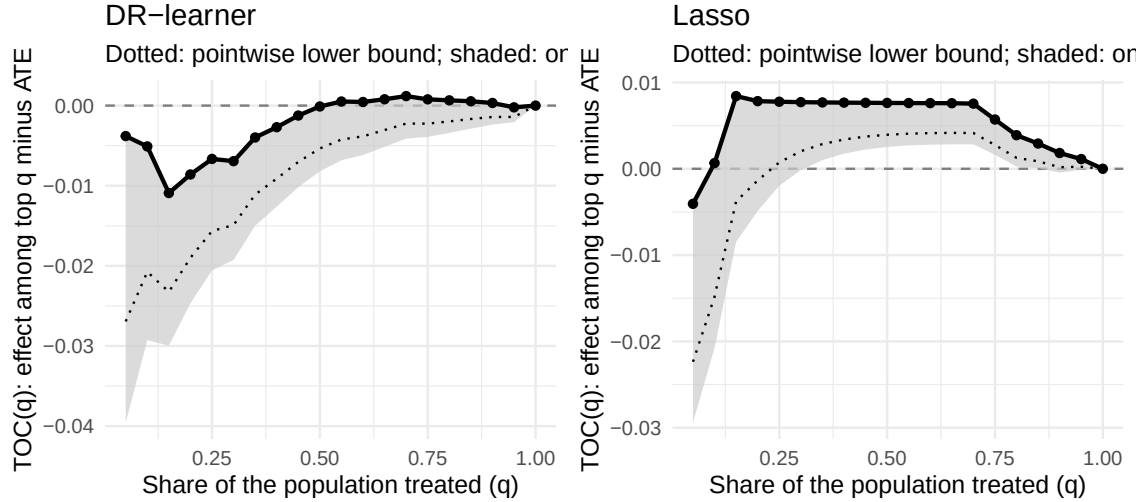


Figure 4: Targeting operator characteristic on the test set for 7 versus 30 days: the DR-learner (left) and the lasso’s implied effect (right).

8.2 A depth-two policy tree on three arms

The paper’s policies are argmax rules over black-box predictions; an interpretable alternative is a shallow tree whose leaves are arms, fitted directly on the doubly robust scores. `policy_learn()` with `engine = "policytree"` runs the exact search of Zhou, Athey, and Wager (2023) on a 60,000-user subsample of the three-arm training scores, over the skill, job, and region dummies; the tree is then priced on the test set with `policy_value()` and against the truth. A tree with four leaves reaches the same true value as the lasso policy, because most of the simulated heterogeneity is carried by skill: the tree gives 7 days to users who are neither experienced nor of mixed skill, 14 days to mixed-skill users, and 30 days to experienced users, with one job-level exception in the experienced branch. It says in three lines who gets what.

```
tree_x <- zcols[grep("skill|job|region", colnames(model.matrix(~ factor(region) + factor(os) + factor(channel) + factor(job) + factor(skill))),
set.seed(3)
sub <- sample.int(nrow(train_d), 60000)
sc_sub <- dr_scores(train_d[sub, ], "subscribed", "arm", zcols, arms = arms, p_hat = design_e(train_d[sub, ]),
learner_mu = lrn("regr.lm"), folds = 2, seed = 1)
pol_tree <- policy_learn(sc_sub, x = tree_x, method = "tree", depth = 2, engine = "policytree", holdout = 0, baseline = "30")
pol_tree
#> Treatment policy (tree, depth 2, policytree, 3 arms)
#> share assigned to each arm: 7: 76%, 14: 13.4%, 30: 10.6%
#> policy_tree object
#> Tree depth: 2
#> Actions: 1: 7 2: 14 3: 30
#> Variable splits:
#> (1) split_variable: z67 split_value: 0
#> (2) split_variable: z69 split_value: 0
#> (4) * action: 1
#> (5) * action: 2
#> (3) split_variable: z56 split_value: 0
#> (6) * action: 3
#> (7) * action: 2
#> expected outcome and gain over the uniform baseline arm:
#> sample n value se gain_vs_baseline se_gain gain_pct share_7
#> estimation 60000 0.1592 0.00369 0.01191 0.003992 8.087 0.7601
#> share_14 share_30
#> 0.1338 0.1061
z_names <- setNames(colnames(model.matrix(~ factor(region) + factor(os) + factor(channel) + factor(job) + factor(skill) + factor(region)),
split_vars <- unique(sapply(Filter(function(nd) !nd$is_leaf, pol_tree$rule$fit$nodes), `[[`, "split_variable"))
z_names[tree_x$split_vars]]
#> z67 z69
```

```

#> "factor(skill)experienced"      "factor(skill)mixed"
#>                                z56
#>    "factor(job)developer"
tree_test <- predict(pol_tree, test_d)
policy_value(sc_test, list(tree = tree_test, lasso = pols_test$lasso), baseline = "30")[, c("policy", "estimate", "std.error", "
#> policy estimate std.error gain_pct share_7 share_14 share_30
#> 1 tree 0.011253292 0.003047543 7.788955 0.7639325 0.1324655 0.10360200
#> 2 lasso 0.009496178 0.003046487 6.572770 0.6516616 0.2557282 0.09261017
c(truth_tree = truth_value(test, tree_test), truth_lasso = truth_value(test, pols_test$lasso))
#> truth_tree truth_lasso
#> 0.1565413 0.1564101

```

9 Lessons

What was replicated. The paper’s procedure, from the descriptive tables through the policy design, the IPS evaluation, and the long-run projections, runs end to end on a population calibrated to the published moments, with the same table and figure layouts. The IPS reward the authors coded is matched exactly by the package’s off-policy value once the within-policy propensities are supplied, and the paper’s ranking of methods, its overfitting warning, and its finding that the subscription policy also wins on retention and revenue all reappear with the true values as a check.

What could not be replicated. The published data are randomized, so no number of the paper can be recomputed from them. The causal tree was replaced by a pruned regression tree on the doubly robust score, and the random forest, boosting, and forest settings were reduced from the authors’ tuned values to keep the note within a reasonable running time.

What the package adds. The multi-arm score object makes the three arms one object, so that the IPS and doubly robust rewards, the pairwise contrasts, the CATE validation, and the policy tree share the same nuisances and the same test set. The doubly robust reward gives the same ranking as IPS, the paired standard error from `policy_value()` replaces the paper’s bootstrap, `cate_validate()` rejects the DR-learner on the test scores before any policy is built on it, and a depth-two policy tree matches the lasso’s true value with an interpretable rule.

10 References

- Yoganarasimhan, H., Barzegary, E., and Pani, A. (2023). Design and evaluation of optimal free trials. *Management Science*, 69(6), 3220–3240.
- Zhou, Z., Athey, S., and Wager, S. (2023). Offline multi-action policy learning: generalization and optimization. *Operations Research*, 71(1), 148–183.
- Kennedy, E. H. (2023). Towards optimal doubly robust estimation of heterogeneous causal effects. *Electronic Journal of Statistics*, 17(2), 3008–3049.
- Athey, S. and Wager, S. (2021). Policy learning with observational data. *Econometrica*, 89(1), 133–161.